

Informe Técnico — EP2: Desarrollo de un Agente Funcional

Asignatura: ISY0101 — Ingeniería de Soluciones con Inteligencia Artificial

Evaluación: Parcial N°2 · Ponderación: 35%

Institución: DuocUC · 2025

Proyecto: Agente de Recursos Humanos — Invermar S.A.

1. Introducción

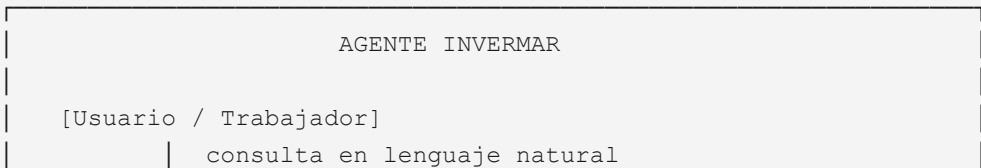
El presente informe documenta el diseño e implementación de un **agente conversacional funcional** para el área de Recursos Humanos de Invermar S.A. y sus empresas relacionadas (Pesquera La Portada S.A., La Península S.A. y Astilleros Calbuco S.A.). Este desarrollo constituye la segunda fase del proyecto iniciado en la Evaluación Parcial N°1, donde se construyó un chatbot reactivo basado en cadenas RAG (*Retrieval-Augmented Generation*).

La diferencia fundamental con la entrega anterior radica en la transición de un sistema de cadenas estáticas (RAG simple) a un **agente autónomo** capaz de razonar, planificar y seleccionar herramientas según el contexto de cada consulta. Esta arquitectura implementa el patrón **ReAct** (*Reasoning + Acting*), que combina razonamiento deliberativo con ejecución adaptativa de acciones (Yao et al., 2023).

Objetivo del agente: responder consultas laborales de trabajadores y del equipo de RRHH sobre liquidaciones, beneficios, finiquitos, vacaciones y normativa laboral chilena, con datos precisos basados en las políticas internas de la empresa y el Código del Trabajo.

2. Arquitectura del Sistema

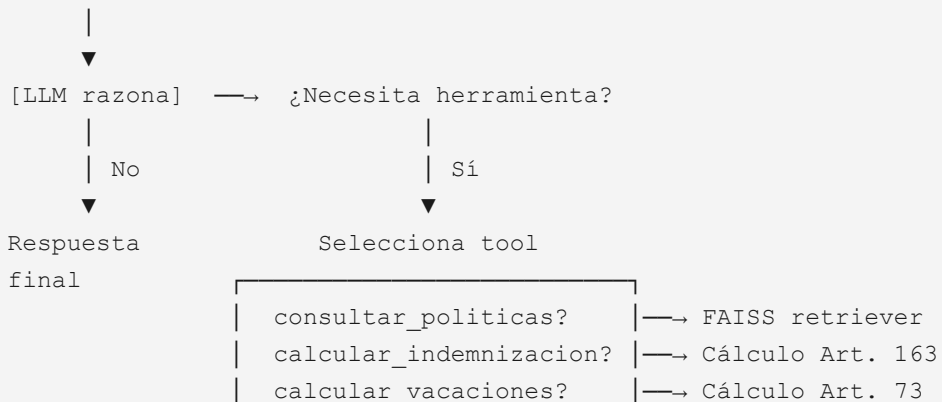
El sistema se organiza en cuatro capas funcionales que interactúan a través del `AgentExecutor` de LangChain:

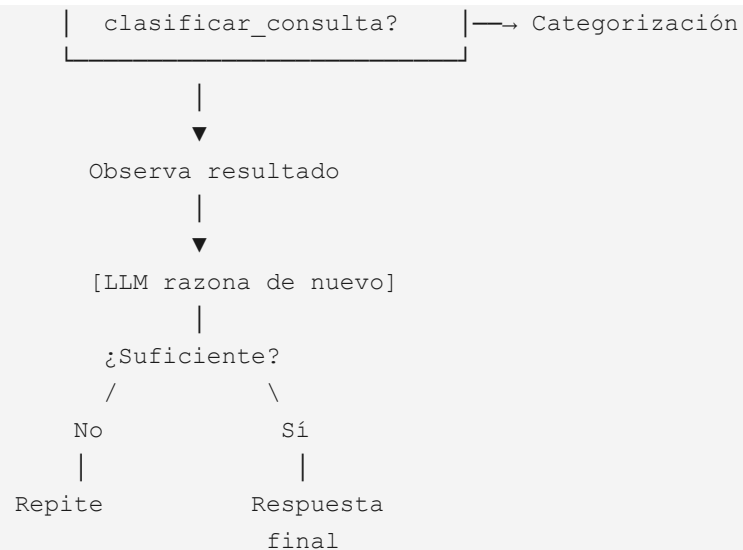




2.1 Flujo de trabajo del agente

Consulta del usuario





3. Implementación de Componentes

3.1 Herramientas del Agente (IE1, IE2)

Las herramientas son las capacidades de acción del agente. Se implementaron con el decorador `@tool` de LangChain, que genera automáticamente el JSON Schema necesario para que el LLM decida cuándo y cómo invocarlas. El *docstring* de cada función es crítico: es la descripción que el modelo lee para elegir la herramienta correcta.

Herramienta	Tipo	Parámetros	Base legal
<code>consultar_politicas_invermar</code>	Consulta RAG	<code>consulta: str</code>	Manual de Beneficios / Políticas internas
<code>calcular_indemnizacion</code>	Cálculo	<code>anios_servicio: int , remuneracion_mensual: int</code>	Art. 163 y 172 Código del Trabajo

Herramienta	Tipo	Parámetros	Base legal
<code>calcular_vacaciones_proporcionales</code>	Cálculo	<pre>meses_trabajados: int, remuneracion_mensual: int</pre>	Art. 73 Código del Trabajo
<code>clasificar_consulta</code>	Razonamiento	<code>consulta: str</code>	Taxonomía interna RRHH

Justificación del framework: Se seleccionó LangChain (`langchain_classic`) por su madurez en la abstracción de agentes ReAct, compatibilidad con el stack del curso (LangChain ≥ 1.0, LangSmith), e integración directa con modelos OpenAI-compatibles vía GitHub Models. Su `AgentExecutor` gestiona automáticamente el ciclo de razonamiento, la ejecución de herramientas y el manejo de errores de parseo, reduciendo el código de orquestación manual.

3.2 Memoria de Corto Plazo – Historial Conversacional (IE3)

La memoria conversacional permite al agente mantener coherencia dentro de una sesión: el trabajador puede hacer preguntas de seguimiento sin repetir el contexto anterior.

Implementación: `WindowChatMessageHistory` extiende `InMemoryChatMessageHistory` con una ventana deslizante de `N = 6` mensajes (3 turnos completos). Al superar el límite, los mensajes más antiguos se descartan automáticamente, controlando el tamaño del contexto enviado al LLM.

```
class WindowChatMessageHistory(InMemoryChatMessageHistory):
    max_messages: int = 6

    def add_message(self, message: BaseMessage) -> None:
        super().add_message(message)
        if len(self.messages) > self.max_messages:
            self.messages = self.messages[-self.max_messages:]
```

El historial se pasa en cada invocación al `AgentExecutor` mediante el parámetro `chat_history`, que el prompt del agente inyecta como `MessagesPlaceholder`. Esto garantiza que el LLM tenga acceso al contexto previo sin sobrecarga de tokens.

3.3 Memoria de Largo Plazo — Recuperación Semántica RAG (IE4)

La memoria de largo plazo almacena el conocimiento **permanente** del agente: políticas y manuales internos de Invermar. A diferencia de la memoria conversacional, esta información persiste y se accede por similitud semántica, no por orden temporal.

Implementación: 8 documentos de políticas internas se vectorizan con `text-embedding-3-small` (OpenAI) y se indexan en **FAISS** (*Facebook AI Similarity Search*), una biblioteca de búsqueda vectorial de alta eficiencia (Johnson et al., 2021). El retriever recupera los 2 fragmentos más similares a cada consulta mediante distancia coseno.

La herramienta `consultar_politicas_invermar` encapsula esta recuperación: el agente la invoca automáticamente cuando detecta una pregunta sobre políticas internas, conectando así ambas capas de memoria en un flujo transparente para el usuario.

3.4 Planificación y Toma de Decisiones (IE5, IE6)

El agente implementa el patrón **ReAct** (Yao et al., 2023): en cada turno, el LLM genera un pensamiento (*Thought*), decide una acción (*Action*), ejecuta la herramienta y observa el resultado (*Observation*), iterando hasta producir la respuesta final.

Esquema de planificación para consultas multi-etapa:

```
Consulta: "Llevo 6 años, me voy en agosto, ¿qué me corresponde en el finiquito?"
|
[Agente razona]
↓ Paso 1: clasificar_consulta → FINIQUITO
↓ Paso 2: calcular_indemnizacion(6, 600000) → $4.200.000
↓ Paso 3: calcular_vacaciones_proporcionales(8, 600000) → $160.000
↓ Síntesis: respuesta integrada con ambos cálculos
```

Esta secuencia no está pre-programada: el agente la infiere del prompt del sistema y del resultado de cada herramienta. Ante una consulta fuera de su dominio (por ejemplo, conflictos interpersonales), clasifica como `OTRO` y deriva al área de RRHH sin inventar información.

4. Resultados y Evidencia de Funcionamiento

Se ejecutaron 5 escenarios de prueba para demostrar la toma de decisiones adaptativa:

Escenario	Herramientas invocadas	Comportamiento observado
Consulta de beneficio (aguinaldo)	<code>consultar_politicas_invermar</code>	Recupera el documento correcto y responde con el monto exacto
Cálculo de indemnización (9 años, \$800.000)	<code>calcular_indemnizacion</code>	Aplica tope de 11 años, calcula \$8.000.000 + aviso previo
Finiquito completo (6 años, agosto)	<code>clasificar_consulta</code> → <code>calcular_indemnizacion</code> → <code>calcular_vacaciones_proporcionales</code>	Secuencia multi-herramienta autónoma
Preguntas de seguimiento (3 turnos)	<code>consultar_politicas_invermar</code> (solo en primera consulta)	El agente recuerda el contexto sin repetir la búsqueda
Consulta fuera de dominio	<code>clasificar_consulta</code>	Detecta categoría <code>OTRO</code> y deriva correctamente

Validación de memoria: se verificó que después de 3 turnos de conversación, el historial contiene exactamente 6 mensajes (ventana llena), y que las respuestas de seguimiento son coherentes con el contexto previo sin que el usuario repita información.

5. Conclusiones

Se implementó exitosamente un agente funcional de RRHH para Invermar S.A. que supera las limitaciones del chatbot RAG de RA1: el sistema ya no ejecuta una cadena estática, sino que **planifica autónomamente** qué herramientas usar en función de la consulta, manteniendo coherencia conversacional mediante dos capas de memoria complementarias.

Los principales aportes técnicos son: (1) la integración de RAG como herramienta del agente en lugar de como cadena independiente, lo que permite combinarla con cálculos legales en un mismo flujo; (2) la separación explícita entre memoria de corto y largo plazo, cada una con propósito y ciclo de vida distintos; y (3) el patrón ReAct como mecanismo de planificación que no requiere programar explícitamente el orden de las herramientas.

Trabajo futuro: integración con API BUK para consultas de liquidaciones reales, persistencia del historial en Redis, autenticación por RUT del colaborador, y despliegue como aplicación web con Streamlit.

Referencias

Chase, H. (2023). *LangChain: Building applications with LLMs through composability*. GitHub. <https://github.com/langchain-ai/langchain>

Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33. <https://arxiv.org/abs/2005.11401>

Ministerio del Trabajo y Previsión Social. (2003). *Código del Trabajo de Chile* (DFL N°1). Biblioteca del Congreso Nacional. <https://www.bcn.cl/leychile/navegar?idNorma=207436>

OpenAI. (2024). *GPT-4o mini: Advancing cost-efficient intelligence*. <https://openai.com/blog/gpt-4o-mini-advancing-cost-efficient-intelligence>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*. <https://arxiv.org/abs/2210.03629>